MODELS OF COMPUTATION | MINI-PROJECT

A DFA for Phishing-URL Pattern Detection

Complete study guide: every state, every transition, the interactive DFA, implementation details, and viva answers

22 states	**14** input symbols
25 transitions	**4** accepting states
8 strings accepted	

What this document is. A single place to learn and explain the DFA you built. Every number, transition, trace and table in it comes from the machine data in `data.js`, so the guide cannot drift from the automaton. Start with the one-minute explanation, then use the state-by-state section when someone points at a circle on the screen and asks “what is this state?”

Contents. 1 The one-minute explanation 2 How the system works 3 Formal definition 4 The complete DFA diagram 5 Transition table and matrix 6 State-by-state details 7 The language the DFA accepts 8 Worked traces 9 Theory points 10 Implementation details 11 Viva questions and answers 12 Last-minute cheat sheet

1. The one-minute explanation

> **Say this first**
> Our project applies Models of Computation to phishing-link detection. We break a URL into tokens (scheme, host type, brand-like word, login/verify action, urgency cue, and so on). Each token class is one letter of a 14-symbol alphabet, so a URL becomes a short string such as SPBAYX. A Deterministic Finite Automaton with 22 states reads that string left to right, one symbol per step, with exactly one possible move each time. If it finishes in one of four accepting states it has reached a verdict: *PhishPaaS*, *PhishBrand*, *PhishMalware* or *BenignPass*. The path it took is the explanation. We implemented and tested it in this interactive study guide.

The four things the DFA can conclude

Verdict state	Plain-English meaning	Accepted pattern (symbols)
q7 PhishPaaS	A brand-like word plus a login/verify action on a free hosting-platform subdomain (the secure-brand-login.vercel.app style).	`S [H] P B A Y X`
q12 PhishBrand	A brand word plus an action on an ordinary host that is not the brand's own domain: brand mismatch.	`S [H] G B A Q X`
q17 PhishMalware	Urgency wording followed by a download/payload lure.	`S [H] G U E R X X`
q19 BenignPass	Ordinary host with a neutral path and none of the risky sequences.	`S [H] G N X`

`[H]` means the optional host-inspection symbol: it may appear once or not at all. Select a row to highlight its state in the diagram.

> **Be upfront about this**
> Our professor's proposal says automata give pattern recognition, not proof that a site is malicious. A match is an indicator. The DFA looks only at the URL text, so it can produce false positives and miss URLs that do not contain the expected tokens. Saying this yourself first is much stronger than being asked.

Why a DFA and not just if-statements?

- **Order matters.** “brand then action” is different from “action then brand”. A state remembers what has already been seen, which a flat keyword list cannot do.
- **Deterministic.** Same input always gives the same path. No guessing, no backtracking.
- **Linear time.** One table lookup per token: time O(n), memory O(1).
- **Explainable.** The state trace (q0 → q1 → q3 → …) shows exactly why a URL was flagged.
- **Formal.** It can be tested, minimised, described by a regular expression and compared with an NFA.

2. How the system works end to end

URL https://secure-brand-login.vercel.app/verify-account	**Normalise and parse** scheme, host labels, path terms
Tokenise each part becomes one token class	**Symbol string** S P B A Y X
DFA run one move per symbol	**Verdict** PhishPaaS

Left to right: the DFA is the fifth box. Everything before it is preparation; everything after it is just reading the final state.

Illustrative tokenisation

The example URL below comes from the proposal document. The mapping is illustrative: it shows the idea, not a claim that the URL is malicious.

URL piece	Token class (symbol)	Meaning
`https://`	**S**	scheme parsed (URL_PARSED)
`*.vercel.app`	**P**	PaaS / hosting-platform host (PLATFORM_HOST)
`brand`	**B**	brand-like token (BRAND_LIKE_TOKEN)
`login`	**A**	credential / verify action (VERIFY_ACTION)
`verify-account`	**Y**	confirming cue: the risk condition is satisfied
`end of URL`	**X**	terminator: nothing more to read

Result: SPBAYX drives the DFA along q0 → q1 → q3 → q4 → q5 → q6 → q7_PhishPaaS, an accepting state, so the verdict is PhishPaaS.

Where this sits in the proposal's four-layer method

Layer	Role	In our DFA?
A: URL lexical / semantic	DFA, NFA and regex over URL tokens	**Yes: this is what we built**
B: Hosting / infrastructure	domain age, hosting class, DNS/certificate	Only the host class (P vs G) is encoded
C: Page / code	HTML/JS inspection	No (optional extension)
D: Behaviour / delivery	redirects and delivery events as a state workflow	No (optional extension)

> **Honest scope**
> Our DFA is a Layer A recogniser. Layers B to D need external data (WHOIS, page fetches, redirect telemetry) and are outside a basic MoC prototype. If asked "does it really detect phishing?", the correct answer is: it recognises suspicious token sequences in the URL, and those matches are signals for a wider risk assessment.

3. Formal definition

A DFA is a 5-tuple `M = (Q, Σ, δ, q0, F)`.

Part	Our value
Q (states)	{ q0, q1, q2, ..., q19, q20, q21 }: 22 states. Four carry names: q7_PhishPaaS, q12_PhishBrand, q17_PhishMalware, q19_BenignPass; q20_Malformed and q21_Trap name the error path.
Σ (alphabet)	`{ S, P, G, H, B, A, Y, X, Q, U, E, R, N, Z }: 14 symbols, each a token class.`
δ (transition function)	Partial function δ : Q × Σ → Q with 25 defined entries (listed in section 5). Undefined entries mean reject.
q0 (start)	q0, marked with the start arrow.
F (accepting)	{ q7_PhishPaaS, q12_PhishBrand, q17_PhishMalware, q19_BenignPass }

Alphabet table (Σ in detail)

Each symbol is a colour-coded chip; select a chip to highlight every transition that uses it in the diagram.

Sym	Name	Meaning as used in our design	Used on
S	Scheme	URL starts with a valid scheme (http / https). Parsing succeeded.	q0
P	PaaS host	Host is a hosting-platform subdomain such as vercel.app, netlify.app, github.io.	q1, q2
G	Generic host	Any other ordinary domain.	q1, q2
H	Host inspection	Optional extra host check token (for example an abnormal subdomain). Allowed once, between S and P/G.	q1
B	Brand-like token	A well-known brand name or lookalike appears.	q3, q8
A	Action token	A credential or verification action (login, verify, secure, account).	q4, q9
Y	PaaS confirmation	Final risk cue on the PaaS path: the configured combination of signals is satisfied.	q5

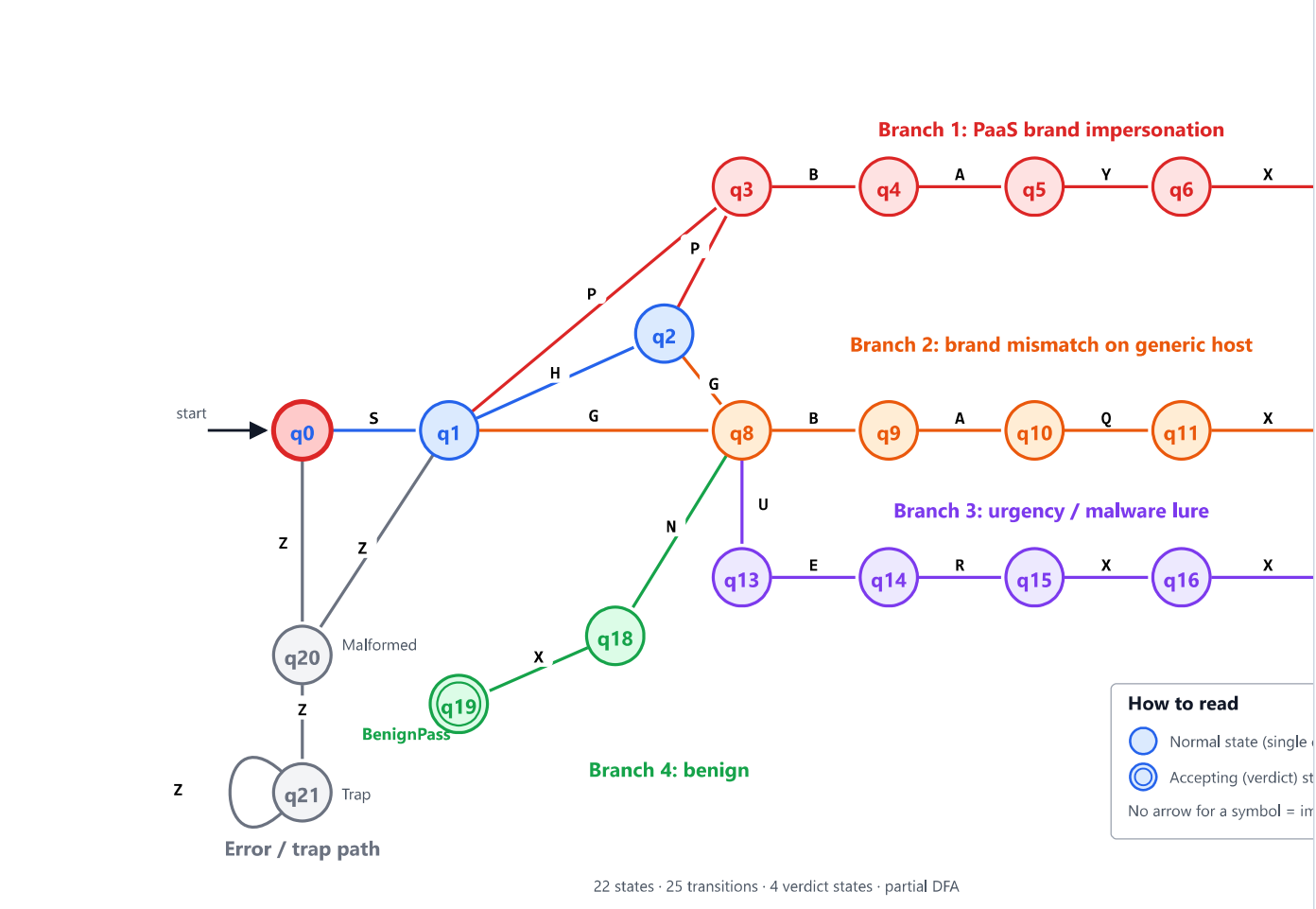
Sym	Name	Meaning as used in our design	Used on
Q	Brand mismatch	Confirms the brand word is not on the brand's real domain.	q10
U	Urgency cue	Pressure wording (urgent, suspended, immediately).	q8
E	Download lure	Executable or download enticement token.	q13
R	Payload / redirect cue	Redirect or payload-delivery cue completing the lure.	q14
N	Neutral token	Ordinary path content with no risky pattern.	q8
X	End marker	Terminator: end of URL / end of the pattern.	q6, q11, q15, q16, q18
Z	Malformed token	Unparseable or invalid piece; sends the run to the error path.	q0, q1, q20, q21

> **Check this table with your teammates**
> The structure (which symbol goes where) comes straight from the code. The wording of what Y, Q, E, R and the second X stand for is my reading of the code comments and the proposal document, because the generator uses only the letters. If your team defined them differently, say so in the viva and use your definitions: the DFA itself does not change.

Each symbol is one character, so any test string is easy to type (for example `SGBAQX`) and each token class has a short label on the arrow.

4. The complete DFA diagram (the centrepiece)

Explore every state and transition. Hover for an explanation, select a state for its full card, or right-click a state for more actions. Use the wheel to zoom and drag to pan.



The diagram uses the supplied state coordinates and transitions. Colours show which branch each state belongs to; the interactive SVG and the simulator use the same 22 states and 25 transitions.

5. Transition table and matrix

5a. Transition matrix δ

Hover a defined cell to highlight its edge in the diagram; click to pin the highlight.

state	S	P	G	H	B	A	Y	Q	U	E
q0 (start)	q1	·	·	·	·	·	·	·	·	·
q1	·	q3	q8	q2	·	·	·	·	·	·
q2	·	q3	q8	·	·	·	·	·	·	·
q3	·	·	·	·	q4	·	·	·	·	·
q4	·	·	·	·	·	q5	·	·	·	·
q5	·	·	·	·	·	·	q6	·	·	·
q6	·	·	·	·	·	·	·	·	·	·
q7 PhishPaaS	·	·	·	·	·	·	·	·	·	·
q8	·	·	·	·	q9	·	·	·	q13	·
q9	·	·	·	·	·	q10	·	·	·	·
q10	·	·	·	·	·	·	·	q11	·	·
q11	·	·	·	·	·	·	·	·	·	·
q12 PhishBrand	·	·	·	·	·	·	·	·	·	·
q13	·	·	·	·	·	·	·	·	·	q14
q14	·	·	·	·	·	·	·	·	·	·
q15	·	·	·	·	·	·	·	·	·	·
q16	·	·	·	·	·	·	·	·	·	·

A dot (·) means no transition is defined: the run stops and the string is rejected. 25 of the 22 × 14 = 308 possible (state, symbol) pairs are defined; the other 283 are implicit rejects. Each pair appears at most once, which is what makes the machine deterministic.

5b. All 25 transitions in plain English

Filter transitions Filter by state, symbol, c

# ↕	From ↕	Read ↕	To ↕	Reading ↕
1	q0	S	q1	Valid scheme read: URL parsing begins.

# ↕	From ↕	Read ↕	To ↕	Reading ↕
2	q1	P	q3	Host is a PaaS platform: go to the PaaS branch.
3	q1	G	q8	Host is a generic domain: go to the generic branch.
4	q1	H	q2	Optional host-inspection token first.
5	q2	P	q3	After inspection, host turned out to be PaaS.
6	q2	G	q8	After inspection, host turned out to be generic.
7	q3	B	q4	Brand-like word seen on the PaaS host.
8	q4	A	q5	Credential/verify action follows the brand.
9	q5	Y	q6	Confirming cue: pattern complete.
10	q6	X	q7 PhishPaaS	End of URL: verdict PhishPaaS.
11	q8	B	q9	Brand-like word on a generic host.
12	q9	A	q10	Credential/verify action follows.
13	q10	Q	q11	Brand mismatch confirmed.
14	q11	X	q12 PhishBrand	End of URL: verdict PhishBrand.
15	q8	U	q13	Urgency cue on a generic host.
16	q13	E	q14	Download/executable lure follows.
17	q14	R	q15	Payload / redirect cue follows.
18	q15	X	q16	First terminator read.
19	q16	X	q17 PhishMalware	Second terminator read: verdict PhishMalware.
20	q8	N	q18	Neutral path content on a generic host.

# ↕	From ↕	Read ↕	To ↕	Reading ↕
21	q18	X	q19 BenignPass	End of URL: verdict BenignPass.
22	q0	Z	q20 Malformed	Malformed token as the very first thing.
23	q1	Z	q20 Malformed	Malformed token right after the scheme.
24	q20	Z	q21 Trap	Second malformed token: fall into the trap.
25	q21	Z	q21 Trap	Trap absorbs any further Z.

6. State-by-state details

Each card answers the questions a teacher usually asks about a circle: what does it mean, how do we get here, where can we go, and what happens on anything else. “Anything else” always means the run is rejected, because the missing transition is an implicit dead end.

Main pipeline: q0, q1, q2

The shared front end every URL passes through before it splits into branches.

q0
initial state
initial state
START

Meaning: Nothing has been read yet. The machine is waiting for the scheme token.
Reached by: Start of every run (start arrow).
Outgoing: S → q1 (scheme read); Z → q20 (malformed from the very start)
Anything else: Reject. A string must begin with S, so something like G... is rejected immediately.

q1
scheme accepted

Meaning: The scheme has been read. This is the first decision hub: the next symbol says what kind of host this is.
Reached by: q0 on S.
Outgoing: P → q3 (PaaS host); G → q8 (generic host); H → q2 (inspect host first); Z → q20
Anything else: Reject. A second S or any path/brand symbol directly after the scheme is invalid.
Note: Only state with four exits. It is where the DFA commits to a branch.

q2
optional host check

Meaning: A host-inspection token has been read. It adds one optional step and then rejoins the normal flow.
Reached by: q1 on H only.
Outgoing: P → q3; G → q8
Anything else: Reject. A second H is not allowed (SHHGNX is rejected at q2).
Note: Placed inside the q1-q3-q8 triangle so no arrow crosses another state. With or without H, the same four patterns are accepted, which is why exactly 8 strings are in the language (4 patterns × 2).

Branch 1: PaaS brand impersonation (q3 to q7)

Hosting-platform host, then brand word, then credential action, then confirmation. Pattern: P B A Y X.

q3
PaaS host seen

Meaning: The host is a hosting-platform subdomain. The DFA now expects a brand-like word.
Reached by: q1 or q2 on P.
Outgoing: B → q4
Anything else: Reject (for example SPAYX: action before brand).

q4
brand on PaaS

Meaning: A brand-like token has appeared on a platform host: the first real suspicion signal.
Reached by: q3 on B.
Outgoing: A → q5
Anything else: Reject.

q5
action after brand

Meaning: Brand plus credential/verify action: the core phishing pattern (the proposal's "pattern recognised" idea).
Reached by: q4 on A.
Outgoing: Y → q6
Anything else: Reject.

q6
awaiting end

Meaning: Pattern is complete; the machine waits for the end marker so it knows nothing else follows.
Reached by: q5 on Y.
Outgoing: X → q7
Anything else: Reject. For example SPBAY stops here without X, so it is rejected even though it looks suspicious.
Note: Not accepting by itself. Requiring X means the whole URL must be consumed.

q7
PhishPaaS
PhishPaaS verdict state
verdict state
ACCEPTING

Meaning: Verdict: PhishPaaS. A brand-impersonation pattern on a PaaS host was recognised in the exact order.
Reached by: q6 on X.
Outgoing: none.
Anything else: Reject. Any extra symbol after q7 is rejected, so the pattern must match the whole input.
Note: The proposal would call this the escalation state ("escalate for verification"), not proof of malice.

Branch 2: brand mismatch on a generic host (q8 to q12)

q8 is the second decision hub. Branch 2 pattern: G B A Q X.

q8
generic host seen

Meaning: The host is an ordinary domain. This is the second hub: the next token chooses brand, urgency or neutral.
Reached by: q1 or q2 on G.
Outgoing: B → q9 (brand); U → q13 (urgency); N → q18 (neutral)
Anything else: Reject.
Note: Three exits, one per remaining branch. q8 is where the generic path becomes three different stories.

Meaning: A brand-like word on a normal domain, which may not be the brand's own site.

q9
brand on generic host

Reached by: q8 on B.
Outgoing: A → q10
Anything else: Reject.

q10
action after brand

Meaning: Brand plus credential action, waiting for the mismatch confirmation.
Reached by: q9 on A.
Outgoing: Q → q11
Anything else: Reject. SGBAX dies here: no mismatch confirmation, so it is not flagged.
Note: Shows that the DFA needs the full chain, not just a brand and an action.

q11
awaiting end

Meaning: Mismatch confirmed; waiting for the end marker.
Reached by: q10 on Q.
Outgoing: X → q12
Anything else: Reject.

q12
PhishBrand
PhishBrand verdict state
verdict state
ACCEPTING

Meaning: Verdict: PhishBrand. Brand word and action on a host that does not belong to the brand.
Reached by: q11 on X.
Outgoing: none.
Anything else: Reject (any extra symbol).

Branch 3: urgency / malware lure (q13 to q17)

From q8 on U. Pattern: G U E R X X.

q13
urgency seen

Meaning: Pressure wording found on a generic host.
Reached by: q8 on U.
Outgoing: E → q14
Anything else: Reject.

q14
lure seen

Meaning: A download / executable enticement follows the urgency.
Reached by: q13 on E.
Outgoing: R → q15
Anything else: Reject.

q15
payload cue

Meaning: Payload or redirect cue completes the lure sequence.
Reached by: q14 on R.
Outgoing: X → q16
Anything else: Reject.

q16
first terminator

Meaning: First terminator read; one more is required.
Reached by: q15 on X.
Outgoing: X → q17
Anything else: Reject. SGUERX (only one X) stops here and is rejected.
Note: This is the only branch that reads X twice. Be ready to say why (see viva question 9).

q17
PhishMalware
PhishMalware verdict state
verdict state
ACCEPTING

Meaning: Verdict: PhishMalware. Urgency, lure and payload cue in the exact order.
Reached by: q16 on X.
Outgoing: none.
Anything else: Reject (any extra symbol).

Branch 4: benign (q18, q19)

From q8 on N. Pattern: G N X. The “nothing suspicious” route.

q18
neutral content

Meaning: Ordinary content on a generic host; none of the risky tokens appeared.
Reached by: q8 on N.
Outgoing: X → q19
Anything else: Reject. SGN without X ends here and is rejected.

q19
BenignPass
BenignPass verdict state
verdict state
ACCEPTING

Meaning: Verdict: BenignPass.
Reached by: q18 on X.
Outgoing: none.
Anything else: Reject (any extra symbol; SGNXS is rejected).
Note: This is an accepting state that means benign. In this machine, accepting = “a definite verdict was reached”, and the state name carries the meaning. It does not mean “phishing”.

Error path: q20, q21

How malformed input is handled instead of silently ignored.

q20
Malformed
Malformed

Meaning: A malformed token has been read, either as the first symbol or right after the scheme.
Reached by: q0 on Z or q1 on Z.
Outgoing: Z → q21
Anything else: Reject.
Note: Not accepting. Strings that reach here can never be accepted.

Meaning: Absorbing state: once here, nothing can lead back to an accepting state.

q21
Trap
Trap
TRAP

Reached by: q20 on Z; itself on Z.

Outgoing: Z → q21 (self-loop)

Anything else: Reject.

Note: Not a fully drawn dead state: only Z loops. Every other symbol is an implicit reject, which has the same effect (see section 9).

7. The language the DFA accepts

```
L(M) = S H? ( P B A Y X | G ( B A Q X | U E R X X | N X ) )
```

#	String	Length	Path	Verdict	
1	SPBAYX	6	q0 → q1 → q3 → q4 → q5 → q6 → q7	PhishPaaS	
2	SHPBAYX	7	q0 → q1 → q2 → q3 → q4 → q5 → q6 → q7	PhishPaaS	
3	SGBAQX	6	q0 → q1 → q8 → q9 → q10 → q11 → q12	PhishBrand	
4	SHGBAQX	7	q0 → q1 → q2 → q8 → q9 → q10 → q11 → q12	PhishBrand	
5	SGUERXX	7	q0 → q1 → q8 → q13 → q14 → q15 → q16 → q17	PhishMalware	
6	SHGUERXX	8	q0 → q1 → q2 → q8 → q13 → q14 → q15 → q16 → q17	PhishMalware	
7	SGNX	4	q0 → q1 → q8 → q18 → q19	BenignPass	
8	SHGNX	5	q0 → q1 → q2 → q8 → q18 → q19	BenignPass	

- Exactly 8 strings are accepted: 2 prefix choices (with or without H) × 4 patterns. Shortest: SGNX (4). Longest: SHGUERXX (8).
- The language is finite, and every finite language is regular. So an equivalent regular expression exists (above).
- Every other string over the 14 symbols is rejected. This is deliberately strict: it recognises exactly the configured patterns, nothing fuzzy.

> **A strength and a weakness, say both**
> Strength: total precision, since every accepted string maps to a named verdict with a clear trace. Weakness: filler tokens are not tolerated. `SG B N A Q X` is rejected. A natural extension is N self-loops on the intermediate states so filler tokens are skipped.

8. Worked traces (accepted and rejected)

Run any row in the simulator, or check every listed input against the expected result and path.

16/16 match expectations

Input	Result	Path (states visited)	Why	Check	
SPBAYX	**ACCEPT**	`q0 → q1 → q3 → q4 → q5 → q6 → q7`	The proposal's example.	✓ **Match**	
SHPBAYX	**ACCEPT**	`q0 → q1 → q2 → q3 → q4 → q5 → q6 → q7`	Same with optional host inspection.	✓ **Match**	
SGBAQX	**ACCEPT**	`q0 → q1 → q8 → q9 → q10 → q11 → q12`	Brand mismatch.	✓ **Match**	
SGUERXX	**ACCEPT**	`q0 → q1 → q8 → q13 → q14 → q15 → q16 → q17`	Malware lure.	✓ **Match**	
SGNX	**ACCEPT**	`q0 → q1 → q8 → q18 → q19`	Benign.	✓ **Match**	
SHGNX	**ACCEPT**	`q0 → q1 → q2 → q8 → q18 → q19`	Benign with host inspection.	✓ **Match**	
SGBAX	**REJECT**	`q0 → q1 → q8 → q9 → q10 (no arrow from q10 on X)`	Missing Q: brand and action but no mismatch confirmation.	✓ **Match**	
SGN	**REJECT**	`q0 → q1 → q8 → q18`	Premature end: stopped in q18, not accepting.	✓ **Match**	
SPBAY	**REJECT**	`q0 → q1 → q3 → q4 → q5 → q6`	Premature end at q6: no X.	✓ **Match**	
SGUERX	**REJECT**	`q0 → q1 → q8 → q13 → q14 → q15 → q16`	Only one X on the malware branch.	✓ **Match**	
SGNXS	**REJECT**	`q0 → q1 → q8 → q18 → q19 (no arrow from q19 on S)`	Extra symbol after a verdict.	✓ **Match**	
GBAQX	**REJECT**	`q0 (no arrow from q0 on G)`	Does not start with S.	✓ **Match**	

Input	Result	Path (states visited)	Why	Check	
SHHGNX	REJECT	q0 → q1 → q2 (no arrow from q2 on H)	Two H symbols.	✓ Match	
Z	REJECT	q0 → q20	Malformed from the start.	✓ Match	
SZ	REJECT	q0 → q1 → q20	Malformed right after scheme.	✓ Match	
SZZZ	REJECT	q0 → q1 → q20 → q21 → q21	Falls into and loops in the trap.	✓ Match	

Two kinds of reject: (1) **dead end:** the run reaches a state with no arrow for the next symbol (for example SGBAX at q10); (2) **ends in a non-accepting state:** the symbols run out too early (for example SGN ends in q18).

9. Theory points a teacher may probe

▶ **1. Is it really a DFA?**

▶ **2. Partial vs complete DFA**

▶ **3. Minimisation**

▶ **4. DFA vs NFA vs regex**

▶ **5. Complexity**

▶ **6. What a DFA cannot do**

10. Implementation details

One source of truth for the DFA

`data.js` stores the 22 states, 25 transitions, 14-symbol alphabet, accepting states, start state, state explanations, branch colours, transition readings, and the 16 test cases. The diagram, matrix, state cards, and simulator all read from this same data.

How the implementation works

- **`index.html`** provides the page shell and loads the local stylesheet and scripts. The app has no framework, server requirement, or external runtime dependency.
- **`style.css`** styles the document, branch-coloured cards, responsive layout, focus states, reduced-motion behavior, and print view.
- **`app.js`** renders the inline SVG and the study-guide sections, connects state/transition interactions, and runs the simulator and illustrative URL tokeniser.
- **`run()`** is a pure transition-table traversal. It returns the verdict, visited-state trace, stopping state, and rejection reason; both the simulator and tests use this same function.
- **URL tokeniser** is a separate, illustrative front end. It turns URL text into symbols, then passes those symbols to the same DFA; the automaton itself remains the formal core.

Run the same DFA logic

The browser console can inspect the result object directly. The unit test script also checks every specified test string and path.

```
const result = PHISHING_DFA.run(
  PHISHING_DFA.DELTA,
  PHISHING_DFA.FINALS,
  "SPBAYX",
  PHISHING_DFA.START
);
console.log(result);
// accepted: true
// trace: [0, 1, 3, 4, 5, 6, 7]
```

To run the tests locally, open a terminal in the project folder and use `node data.test.js`. The result should report that all 16 strings match their expected results and paths.

11. Viva questions and answers

Search questions and answers

☐ Practice mode (hide answers)

▶ **1. What is a DFA, in one sentence?**

▶ **2. Why did you choose a DFA for phishing detection?**

▶ **3. What is your alphabet and why single letters?**

▶ **4. How many states, transitions, accepting states?**

▶ **5. What does each accepting state mean?**

▶ **6. Is it a complete DFA? You have missing transitions.**

▶ **7. Which strings does it accept?**

▶ **8. Is the language finite? Does that matter?**

▶ **9. Why does the malware branch read X twice?**

▶ **10. What is q2 for?**

▶ **11. What happens to malformed input?**

▶ **12. What happens if there are extra symbols after a verdict?**

▶ **13. Prove it is deterministic.**

▶ **14. Could you minimise it? Why didn't you?**

▶ **15. How is this different from an NFA?**

▶ **16. What is the time complexity?**

▶ **17. Does a match prove the URL is phishing?**

▶ **18. What are the limitations?**

▶ **19. How would you improve it?**

▶ **20. How does it relate to the ML paper in your review sheet?**

▶ **21. Why are q2, q18 and q19 placed where they are in the diagram?**

▶ **22. How is the interactive implementation organized?**

▶ **23. What is the proposal's 5-state example compared to your 22 states?**

12. Last-minute cheat sheet

Item	Value
Machine	DFA (partial), M = (Q, Σ, δ, q0, F)
Numbers	22 states, 14 symbols, 25 transitions, 1 start, 4 accepting, 8 accepted strings
Alphabet	S scheme · P PaaS host · G generic host · H host inspection · B brand · A action · Y PaaS confirm · Q mismatch · U urgency · E lure · R payload cue · N neutral · X end · Z malformed
Language	S H? (P B A Y X | G (B A Q X | U E R X X | N X))
Verdicts	q7 PhishPaaS · q12 PhishBrand · q17 PhishMalware · q19 BenignPass
Hubs	q1 (host type: P / G / H) and q8 (after generic host: B / U / N)
Error path	Z: q0 or q1 → q20 → q21 (loops on Z), never accepting
Partial DFA	283 of 308 pairs undefined = implicit reject; complete by sending them to q21
Minimisation	15 classes incl. dead; merges q6/q11/q16/q18, the four finals and q20/q21; kept separate for verdict labels
Complexity	O(n) time, O(1) space
Implementation	Offline single-page app in plain HTML, CSS and vanilla JavaScript
Test strings	Accept: SPBAYX, SHPBAYX, SGBAQX, SGUERXX, SGNX, SHGNX. Reject: SGBAX, SGN, SPBAY, SGUERX, SGNXS, GBAQX, SHHGNX, Z, SZ, SZZZ
Always say	It recognises suspicious token sequences in the URL. A match is an indicator, not proof of phishing.